



UNIT 4: HASHING AND FILE STRUCTURES

Q.1. What is Hashing in DS? Explain in brief.

Introduction

Hashing is a technique in **Data Structures** used to **store and retrieve data quickly**. It transforms a given key (data item) into an index of an array using a special function called a **hash function**.

The data is stored in a structure called a **hash table**, where each position is identified by an index.

☞ The main goal of hashing is to achieve **fast searching, insertion, and deletion**, ideally in **O(1) time complexity**.

Hash Function

A **hash function** is a mathematical function that converts a key into an index.

Example:

$$h(\text{key}) = \text{key} \bmod m$$

Where:

- **key** = input value
- **m** = size of hash table

Working of Hashing (Step-by-Step)

1. Take a key (e.g., student ID or number).
2. Apply the hash function to generate an index.
3. Store the value at that index in the hash table.
4. To search, apply the same hash function to find the location.

Diagram of Hashing

Keys: 101, 102, 105, 115

Hash Function:

$$h(\text{key}) = \text{key} \bmod 10$$

◇ Step 1: Calculate Index

- $h(101) = 1$
- $h(102) = 2$
- $h(105) = 5$
- $h(115) = 5$ ☞ Collision here

◇ Step 2: Final Hash Table (Using Chaining)

Index Values

0	-
1	101
2	102
3	-
4	-
5	105 → 115
6	-
7	-
8	-
9	-

Diagram Representation

Index 1 → [101]

Index 2 → [102]

Index 5 → [105] → [115]

☞ Each key is directly mapped to a specific index, making access very fast.

Collision in Hashing

Sometimes, two different keys produce the **same index**. This situation is called a **collision**.

Example:

$$h(105) = 105 \% 10 = 5$$

$$h(115) = 115 \% 10 = 5$$

☞ Both values map to index **5**, causing a collision.

Basic Collision Handling Methods

1. **Separate Chaining** – Store multiple elements at the same index using a linked list.
2. **Open Addressing** – Find another empty slot (Linear probing, Quadratic probing, Double hashing).

Advantages of Hashing

- Very fast data access ($O(1)$ average time)
- Efficient for large data
- Easy implementation

Disadvantages of Hashing

- Collision handling required
- Poor hash function can reduce performance
- Fixed table size (may cause overflow)

Applications of Hashing

- Database indexing
- Symbol tables in compilers
- Password storage
- Caching systems

Conclusion

Hashing is an efficient technique that maps data to specific locations using a hash function, allowing quick operations. Although collisions may occur, proper techniques can handle them effectively.

Q.2. What are the different types of collision resolution techniques in hashing? Explain each method with suitable examples.

Introduction

A **collision** occurs when two or more keys are mapped to the same index in a hash table.

Collision resolution techniques are broadly divided into:

1. Separate Chaining
2. Open Addressing

1. Separate Chaining (Open Hashing)

Definition

Each index of the hash table contains a **linked list**. When collision occurs, new elements are added to the list at that index.

Example:

Keys: 101, 102, 112

Hash Function:

$h(\text{key}) = \text{key} \bmod 10$

- $h(101) = 1$
- $h(102) = 2$
- $h(112) = 2 \rightarrow$ Collision

Hash Table:

Index Values

1 101

2 102 $\rightarrow$ 112

Advantages:

- Simple and easy to implement
- Table never gets full

Disadvantages:

- Extra memory required for linked list

2. Open Addressing (Closed Hashing)

Definition

In this method, all elements are stored **within the hash table itself**. When collision occurs, we find another empty position using probing techniques.

Types of Open Addressing

1. Linear Probing (Open Addressing)

Concept:

If a collision occurs, check the **next available position** sequentially.

Formula:

$$h_i(\text{key}) = (h(\text{key}) + i) \bmod m$$

Example:

Keys: **101, 111**

- $h(101) = 1 \rightarrow$ stored at index 1
- $h(111) = 1 \rightarrow$ collision $\rightarrow$ store at index 2

Hash Table:

Index Value

1 101

2 111

Advantages:

- No extra memory needed
- Easy to implement

Disadvantages:

- **Clustering problem** (grouping of elements)

2. Quadratic Probing

Concept:

Instead of linear steps, the next position is calculated using **square values (i^2)**.

Formula:

$$h(\text{key}) = (h(\text{key}) + i^2) \bmod m$$

Example:

Keys: **101, 111**

- $h(101) = 1 \rightarrow$ stored at index 1
- $h(111) = 1 \rightarrow$ collision
 - $i=1 \rightarrow (1+1^2)=2 \rightarrow$ store at index 2

Hash Table:

Index Value

1 101

2 111

Advantages:

- Reduces clustering compared to linear probing

Disadvantages:

- More complex than linear probing
- May not use all table positions

3. Double Hashing

Concept:

Uses **two hash functions** to calculate new positions.

Formula:

$$h(\text{key}) = (h1(\text{key}) + i * h2(\text{key})) \bmod m$$

Example:

Keys: **105, 115**

- $h_1(105) = 5 \rightarrow$ stored at 5
- $h_1(115) = 5 \rightarrow$ collision
- $h_2(115) = 4$
- New index = $5 + 4 = 9$

Hash Table:

Index Value

5 105

9 115

Advantages:

- Minimizes clustering
- More efficient

Disadvantages:

- More computation required

Conclusion

- **Separate Chaining** $\rightarrow$ Uses linked list
- **Open Addressing** $\rightarrow$ Uses probing methods
 - Linear Probing
 - Quadratic Probing
 - Double Hashing

☞ **Chaining is simplest, while Double Hashing is most efficient.**

Q.3. Differentiate between Sequential, Indexed, and Relative (Random) file organization methods. Illustrate with appropriate use cases and diagrams.

Introduction

File organization refers to the method of arranging records in a file so that they can be **stored, accessed, and updated efficiently**.

The three common methods are **Sequential, Indexed, and Relative (Random)** file organization.

1. Sequential File Organization

Definition

In this method, records are stored **one after another in a fixed order**, usually based on a key field.

Diagram

[101] → [102] → [103] → [104] → [105]

Explanation

- Records are arranged in **sorted order**.
- To find a record, the system must **search from the beginning** until the desired record is found.
- Insertion and deletion are difficult because records must maintain order.

Advantages

- Simple to implement
- Suitable for batch processing

Disadvantages

- Slow searching
- Not suitable for frequent updates

Use Case

- Payroll systems
- Report generation

2. Indexed File Organization

Definition

This method uses a **separate index file** that stores the key and the address of records for fast access.

Diagram

Index:

101 → 0

102 → 1

103 → 2

Data:

[101] [102] [103]

Explanation

- An index acts like a **table of contents**.
- It allows **direct access** to records without scanning the entire file.
- Supports both sequential and random access.

Advantages

- Faster searching
- Efficient for large files

Disadvantages

- Requires extra storage for index
- More complex than sequential

Use Case

- Database systems
- Library management systems

3. Relative (Random) File Organization

Definition

Records are stored at positions calculated using a **hash function**, allowing **direct (random) access**.

Diagram

$h(\text{key}) = \text{key} \bmod 10$

Index Record

1	101
2	102
5	105

Explanation

- The location of each record is determined using a **hash function**.
- No need to search sequentially.
- Collisions may occur and must be handled.

Advantages

- Very fast access
- Suitable for real-time applications

Disadvantages

- Collision problem
- More complex implementation

Use Case

- Banking systems
- Online transaction systems

Difference Table (Important ★)

Feature	Sequential	Indexed	Relative (Random)
Storage	Ordered	With index	Using hash
Access Method	Sequential	Direct + Sequential	Direct
Speed	Slow	Fast	Very Fast
Extra Memory	No	Yes (index)	Moderate
Complexity	Low	Medium	High
Updates	Difficult	Easy	Easy

Conclusion

- **Sequential** is simple but slow.
- **Indexed** improves searching using an index.

- **Relative** provides fastest access using hashing.

☞ Each method is used based on system requirements.